



UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

INTEGRANTES:

BEDON VITERI KEYLA
CASTRO LOPEZ LEONARDO
EMANUEL PINO JULIANA
LUNA MORA ERNESTO

ING. GUERRERO ULLOA GLEISTON CICERON

CARRERA:

7MO SOFTWARE A

MATERIA:

APLICACIONES DISTRIBUIDAS

AÑO LECTIVO

2026-2027

PRACTICA 1

Implementación TCP

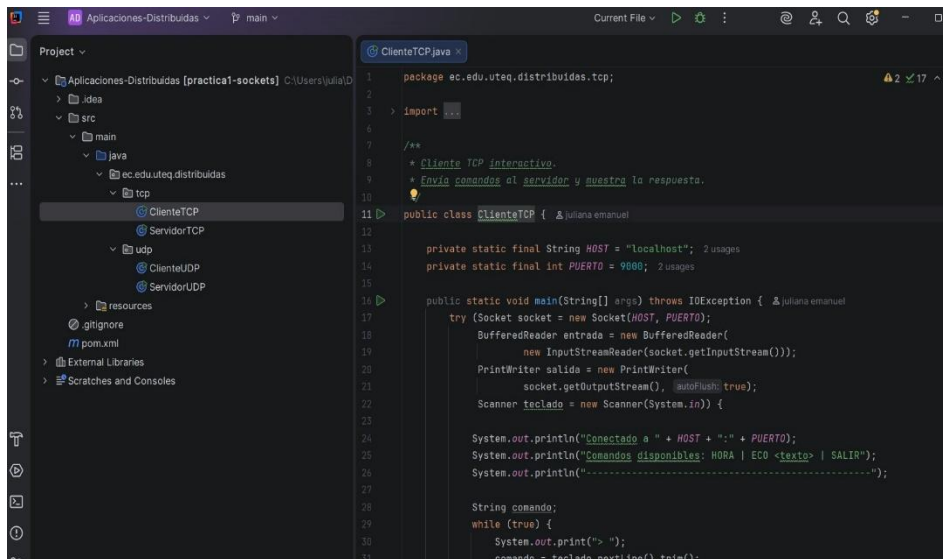
El cliente TCP (ClienteTCP) se conecta al servidor en localhost, puerto 9000. Al establecer la conexión, muestra los comandos disponibles y entra en un bucle interactivo esperando entrada del usuario.

Comandos soportados:

HORA: solicita al servidor la hora actual del sistema.

ECO <texto>: envía texto al servidor y lo recibe de vuelta con un contador de mensajes.

SALIR: cierra la conexión con el servidor de forma ordenada.



```
1 package ec.edu.uteq.distribuidas.tcp;
2
3 import java.io.*;
4
5 /**
6  * Cliente TCP interactivo.
7  * Envía comandos al servidor y muestra la respuesta.
8  */
9
10 public class ClienteTCP {
11     private static final String HOST = "localhost";
12     private static final int PUERTO = 9000;
13
14     public static void main(String[] args) throws IOException {
15         try (Socket socket = new Socket(HOST, PUERTO);
16             BufferedReader entrada = new BufferedReader(
17                 new InputStreamReader(socket.getInputStream()));
18             PrintWriter salida = new PrintWriter(
19                 socket.getOutputStream(), true);
20             Scanner teclado = new Scanner(System.in)) {
21             System.out.println("Conectado a " + HOST + " - " + PUERTO);
22             System.out.println("Comandos disponibles: HORA | ECO <texto> | SALIR");
23             System.out.println("-----");
24
25             String comando;
26             while (true) {
27                 System.out.print("> ");
28                 comando = teclado.nextLine().trim();
29             }
30         }
31     }
32 }
```

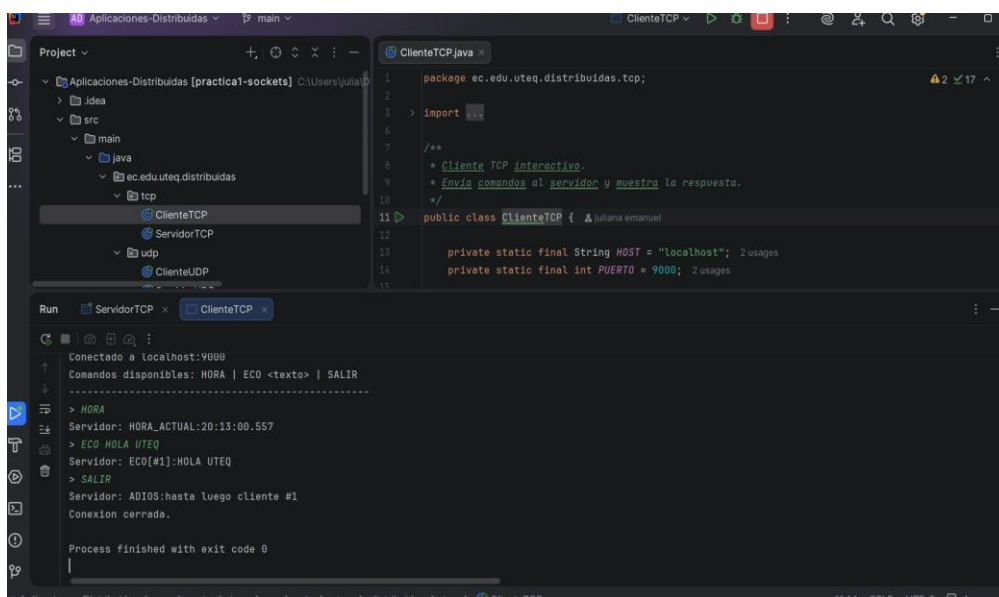
Ejecución del Cliente TCP

La siguiente captura muestra la ejecución simultánea del ServidorTCP y el ClienteTCP. El cliente se conectó exitosamente al servidor en localhost:9000 y ejecutó los tres comandos disponibles:

HORA: el servidor respondió con la hora actual (20:13:00.557).

ECO HOLA UTEQ: el servidor devolvió el mensaje con el contador ECO[#1]:HOLA UTEQ.

SALIR: el servidor respondió con ADIOS:hasta luego cliente #1 y cerró la conexión.



```
Conectado a localhost:9000
Comandos disponibles: HORA | ECO <texto> | SALIR
-----
> HORA
Servidor: HORA_ACTUAL:20:13:00.557
> ECO HOLA UTEQ
Servidor: ECO[#1]:HOLA UTEQ
> SALIR
Servidor: ADIOS:hasta luego cliente #1
Conexion cerrada.

Process finished with exit code 0
```

Implementación y Ejecución del Cliente UDP

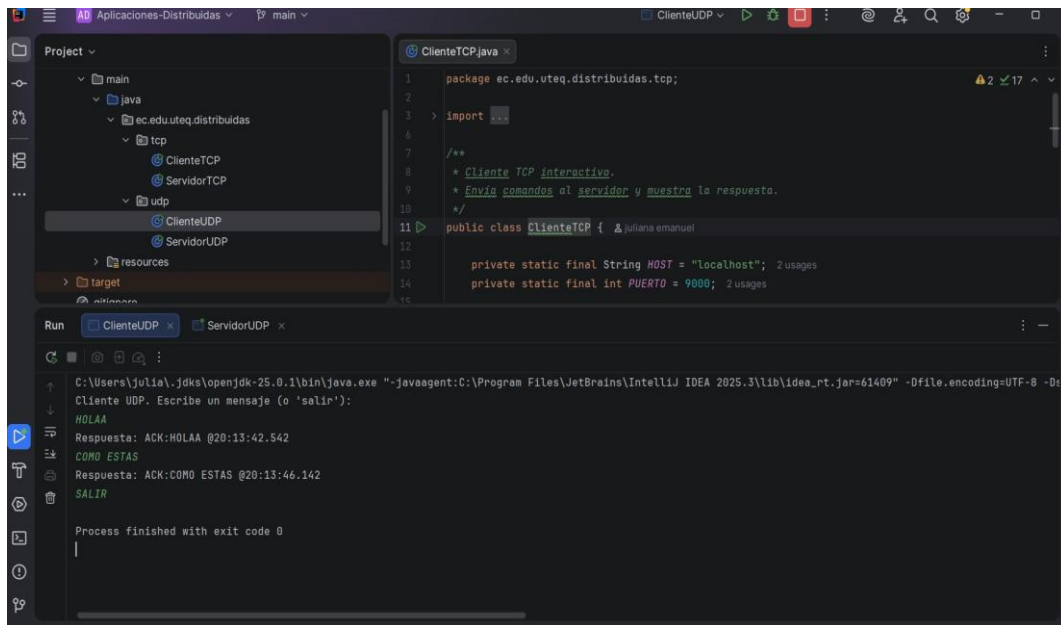
El cliente UDP (ClienteUDP) utiliza el protocolo UDP para enviar mensajes al servidor. A diferencia de TCP, UDP no establece una conexión persistente; cada mensaje se envía de forma independiente. El servidor responde con un ACK (acuse de recibo) que incluye el mensaje y la marca de tiempo.

La ejecución muestra el siguiente comportamiento:

Mensaje HOLAA: el servidor respondió con ACK:HOLAA @20:13:42.542.

Mensaje COMO ESTAS: el servidor respondió con ACK:COMO ESTAS @20:13:46.142.

Mensaje SALIR: el cliente terminó la sesión normalmente (exit code 0).



PRACTICA 2

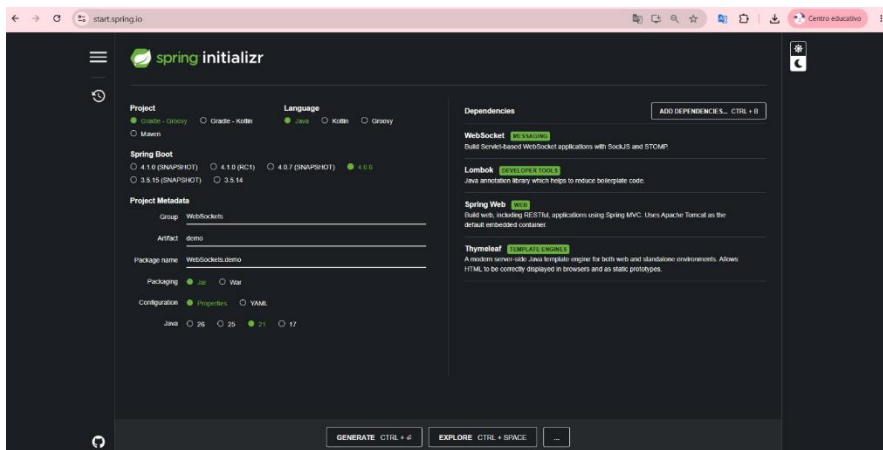
Se muestra la configuración inicial de un proyecto de software utilizando Spring Initializr, una herramienta para generar la estructura base de aplicaciones basadas en el framework Spring Boot.

Aquí tienes un resumen de lo que se está configurando:

Configuración del Proyecto

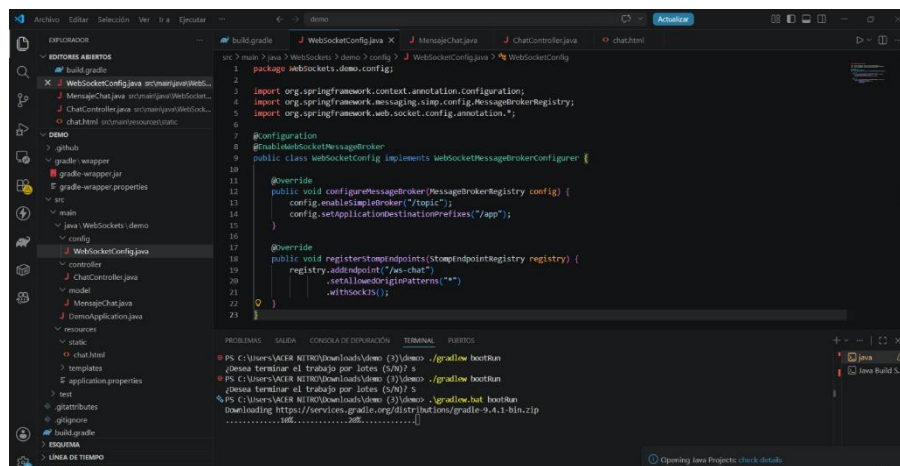
Herramienta de construcción: Se ha seleccionado Gradle - Groovy para gestionar las dependencias y el ciclo de vida del proyecto.

- **Lenguaje:** El proyecto se desarrollará utilizando Java (versión 21).
- **Versión de Spring Boot:** Se está utilizando la versión 4.0.6.
- **Metadatos:** El proyecto tiene el nombre de grupo WebSockets y el artefacto se llama demo.



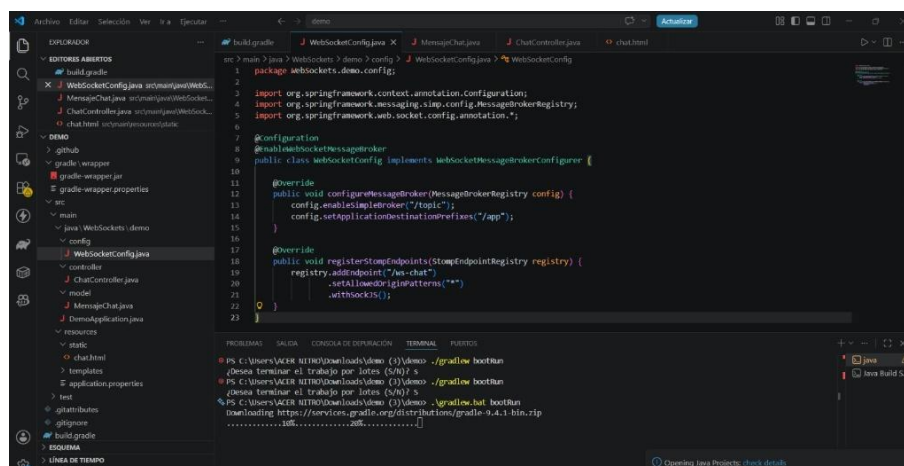
En esta etapa, estás pasando de la configuración inicial en la web a la estructura de desarrollo real dentro de un editor de código (VS Code). Estás organizando la arquitectura del microservicio siguiendo el patrón de diseño MVC (Modelo-Vista-Controlador).

Aquí te explico la organización de las carpetas y archivos que se ven en la imagen:

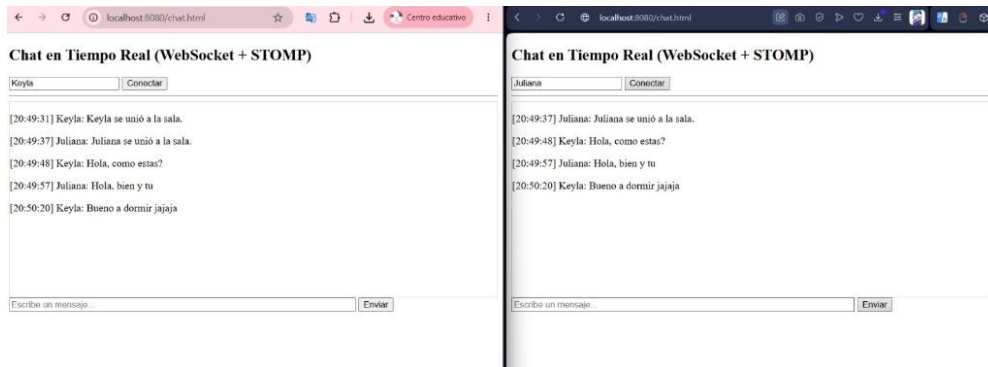


Lo que estás viendo en la terminal es el proceso de aprovisionamiento y construcción del entorno de ejecución mediante Gradle.

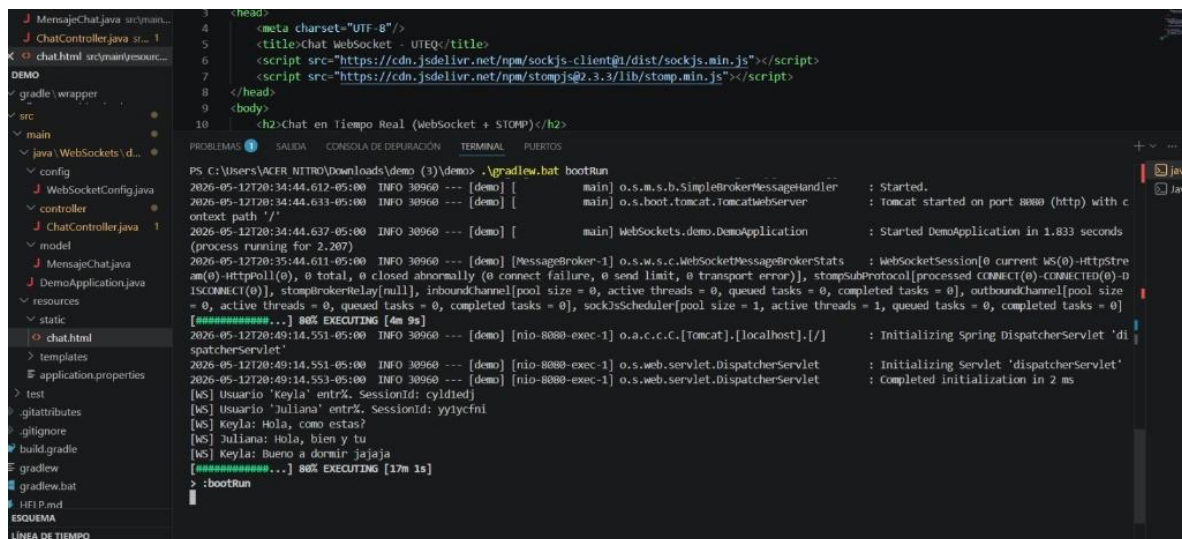
Aquí tienes un texto que explica detalladamente qué está ocurriendo:



Finalmente, logramos poner en marcha la aplicación y comprobar que el chat funciona correctamente en tiempo real. En la imagen se puede ver cómo abrimos dos ventanas del navegador por separado para simular la interacción entre dos usuarios distintos, utilizando el protocolo WebSocket + STOMP.



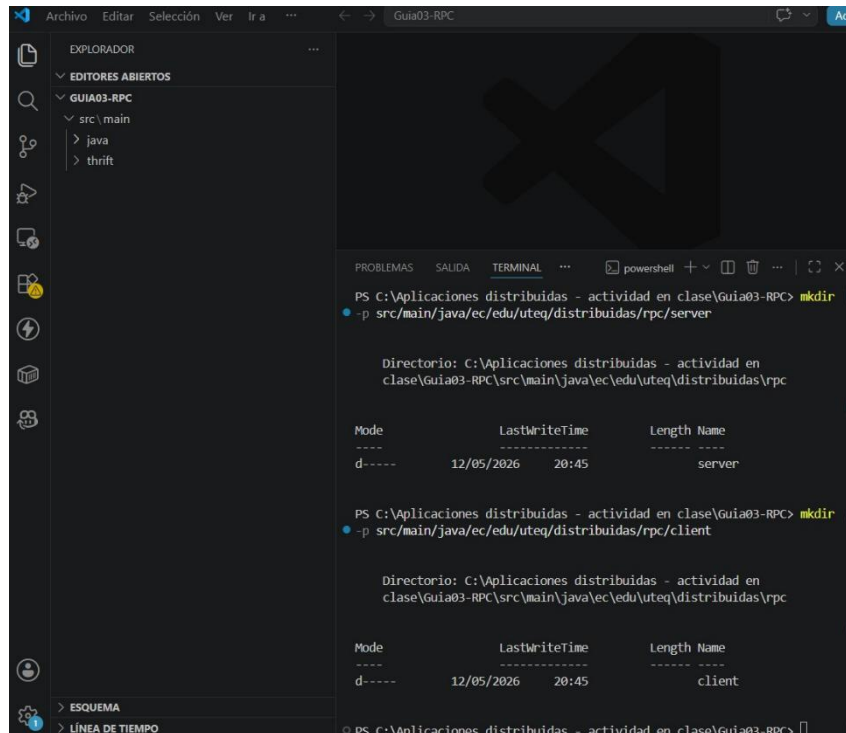
En esta parte del proceso, ya tenemos el servidor Tomcat levantado y funcionando en el puerto 8080, tal como se ve en los logs de la terminal. Lo más importante es que podemos confirmar que la comunicación por WebSockets está activa, ya que en la consola se registran en tiempo real las interacciones que estamos teniendo en el chat.



PRACTICA 3

Para avanzar con la actividad de Aplicaciones Distribuidas, como grupo empezamos a armar la estructura de carpetas necesaria para trabajar con RPC. Lo que hicimos fue usar la terminal para crear de una vez los directorios de server y client dentro del paquete del proyecto en Java.

Básicamente, estamos preparando el terreno para separar la lógica de lo que va a hacer el servidor y lo que va a manejar el cliente, asegurándonos de que todo el proyecto esté bien organizado desde el inicio.



The screenshot shows an IDE window titled 'Guia03-RPC'. On the left, the 'EXPLORADOR' (Explorer) pane shows the project structure: 'EDITORES ABIERTOS' (Open Editors) with 'GUIA03-RPC' and 'src/main' containing 'java' and 'thrift'. The main editor area shows the 'TERMINAL' output of two 'mkdir' commands. The first command creates the 'server' directory, and the second creates the 'client' directory. Both commands are executed from the path 'C:\Aplicaciones distribuidas - actividad en clase\Guia03-RPC\src\main\java\ec\edu\uteq\distribuidas\rpc'. The terminal output shows the directory paths and the creation of the 'server' and 'client' directories.

```
PS C:\Aplicaciones distribuidas - actividad en clase\Guia03-RPC> mkdir -p src/main/java/ec/edu/uteq/distribuidas/rpc/server

Directorio: C:\Aplicaciones distribuidas - actividad en clase\Guia03-RPC\src\main\java\ec\edu\uteq\distribuidas\rpc

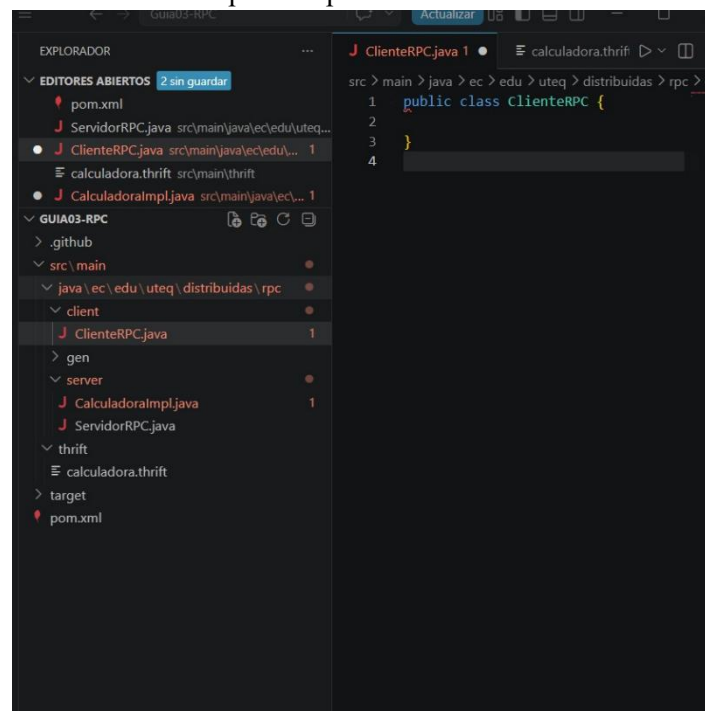
Mode                LastWriteTime         Length Name
----                -
d-----          12/05/2026   20:45             server

PS C:\Aplicaciones distribuidas - actividad en clase\Guia03-RPC> mkdir -p src/main/java/ec/edu/uteq/distribuidas/rpc/client

Directorio: C:\Aplicaciones distribuidas - actividad en clase\Guia03-RPC\src\main\java\ec\edu\uteq\distribuidas\rpc

Mode                LastWriteTime         Length Name
----                -
d-----          12/05/2026   20:45             client
```

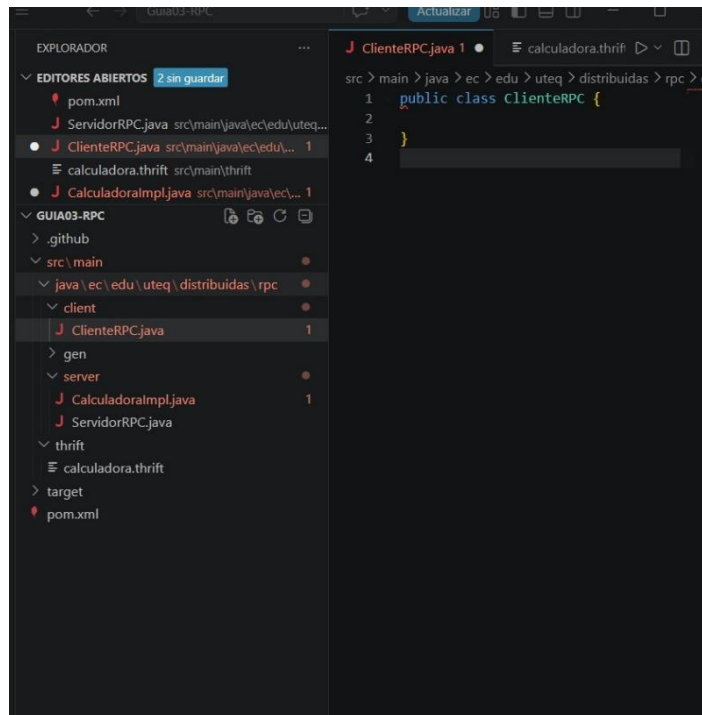
En esta etapa del trabajo grupal, hemos avanzado con la creación de los archivos Java necesarios dentro de la estructura previamente organizada para el proyecto Guia03-RPC. En el explorador de archivos se observa que ya definimos la lógica principal: el archivo ClienteRPC.java en la carpeta de cliente, y los archivos ServidorRPC.java junto con CalculadoraImpl.java en la carpeta de servidor para gestionar las peticiones y procesos. Con esto, ya tenemos vinculados los componentes funcionales con nuestra definición de calculadora.thrift, dejando el entorno listo para las pruebas de comunicación remota.



The screenshot shows the IDE with the 'EXPLORADOR' (Explorer) pane on the left and the 'CLIENTE RPC.java' file open in the main editor. The Explorer pane shows the project structure: 'EDITORES ABIERTOS' (Open Editors) with 'pom.xml', 'ServidorRPC.java', 'CalculadoraImpl.java', and 'Calculadora.thrift'. The 'GUIA03-RPC' folder contains '.github', 'src/main', 'java', 'ec', 'edu', 'uteq', 'distribuidas', 'rpc', 'client', 'gen', 'server', 'thrift', and 'target'. The 'client' folder contains 'ClienteRPC.java'. The 'server' folder contains 'CalculadoraImpl.java' and 'ServidorRPC.java'. The 'thrift' folder contains 'calculadora.thrift'. The main editor shows the code for 'ClienteRPC.java'.

```
src > main > java > ec > edu > uteq > distribuidas > rpc > client
1 public class ClienteRPC {
2
3 }
4
```

Para esta fase final del proyecto Guia03-RPC, como grupo completamos la estructura de archivos en image_713f43.jpg, integrando la lógica en las carpetas de cliente y servidor. Es importante resaltar que la carpeta gen se mantuvo vacía inicialmente, ya que el código de soporte se genera de forma automática al ejecutar el compilador a partir de nuestra definición en calculadora.thrift. Con esta configuración lista, logramos que todos los componentes se vinculen correctamente, permitiendo que la arquitectura de comunicación remota funcione según lo previsto.



Este es el comando donde se hace automático los archivos

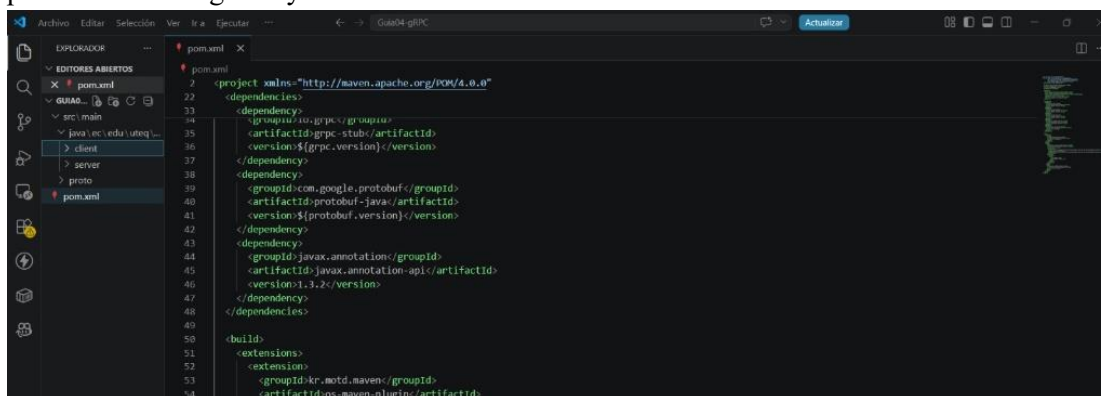
```
mvn clean compile
[INFO] Scanning for projects...
[INFO] -----< ec.edu.uteq.distribuidas:guia03-rpc >-----
[INFO] Building guia03-rpc 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- clean:3.2.0:clean (default-clean) @ guia03-rpc ---
[INFO] Deleting C:\Aplicaciones distribuidas - actividad en clase\Guia03-RPC\target
[INFO] --- resources:3.3.1:resources (default-resources) @ guia03-rpc ---
[INFO] skip non existing resourceDirectory C:\Aplicaciones distribuidas - actividad en clase\Guia03-RPC\src\main\resources
[INFO] --- compiler:3.13.0:compile (default-compile) @ guia03-rpc ---
```

En esta etapa final, logramos ejecutar con éxito el servidor de nuestro proyecto Guia03-RPC utilizando Maven. Como se muestra en la terminal, el servidor Thrift ha iniciado correctamente y se encuentra escuchando peticiones en el puerto 9090.

```
PS C:\Aplicaciones distribuidas - actividad en clase\Guia03-RPC> mvn exec:java "-Dexec.mainClass=ec.edu.uteq.distribuidas.rpc.server.ServidorRPC"
[INFO] -----< ec.edu.uteq.distribuidas:guia03-rpc >-----
[INFO] Building guia03-rpc 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- exec:3.1.0:java (default-cli) @ guia03-rpc ---
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Servidor Thrift (RPC) escuchando en puerto: 9090
RPC sumar(5,00, 3,00)
RPC restar(10,00, 4,00)
RPC multiplicar(6,00, 7,00)
RPC dividir(15,00, 4,00)
RPC dividir(10,00, 0,00)
```

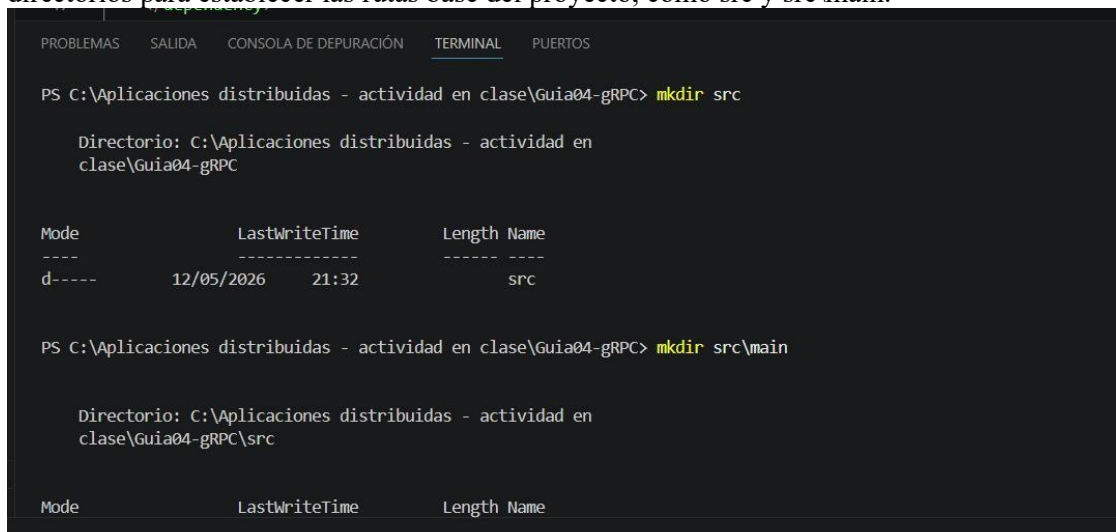
PRACTICA 4

Estamos trabajando en la configuración inicial de un nuevo proyecto llamado Guia04-gRPC. Lo que estamos haciendo aquí es editar el archivo pom.xml para incluir todas las dependencias necesarias que nos permitan utilizar gRPC y Protocol Buffers en Java.



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  <dependencies>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-stub</artifactId>
      <version>${grpc.version}</version>
    </dependency>
    <dependency>
      <groupId>com.google.protobuf</groupId>
      <artifactId>protobuf-java</artifactId>
      <version>${protobuf.version}</version>
    </dependency>
    <dependency>
      <groupId>javax.annotation</groupId>
      <artifactId>javax.annotation-api</artifactId>
      <version>1.3.2</version>
    </dependency>
  </dependencies>
  <build>
    <extensions>
      <extension>
        <groupId>kr.motd.maven</groupId>
        <artifactId>no-maven-plugin</artifactId>
```

Para continuar con el desarrollo de la Guía 04-gRPC, como grupo estamos organizando la estructura de carpetas necesaria mediante la terminal. En este paso, estamos ejecutando comandos de creación de directorios para establecer las rutas base del proyecto, como src y src/main.



```
PS C:\Aplicaciones distribuidas - actividad en clase\Guia04-gRPC> mkdir src

Directorio: C:\Aplicaciones distribuidas - actividad en
clase\Guia04-gRPC

Mode                LastWriteTime         Length Name
----                -
d-----         12/05/2026    21:32         src

PS C:\Aplicaciones distribuidas - actividad en clase\Guia04-gRPC> mkdir src\main

Directorio: C:\Aplicaciones distribuidas - actividad en
clase\Guia04-gRPC\src

Mode                LastWriteTime         Length Name
```

En esta última etapa, logramos poner en marcha el servidor de nuestro proyecto Guia04-gRPC. Como se muestra en la terminal, al ejecutar el comando de Maven, el sistema descargó las dependencias finales necesarias para el entorno de ejecución.

```
PROBLEMAS 11 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Aplicaciones distribuidas - actividad en clase\Guia04-grpc> mvn exec:java "-Dexec.mainClass=ec.edu.uteq.distribuidas.grpc.server.ServidorGRPC"
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.14.1/junit-bom-5.14.1.pom (5.7 kB at 64 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.9.1/asm-9.9.1.pom (2.4 kB at 25 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/9.9.1/asm-commons-9.9.1.pom (2.8 kB at 30 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/9.9.1/asm-tree-9.9.1.pom (2.6 kB at 29 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.24/maven-resolver-util-1.9.24.jar (196 kB at 1.7 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.24/maven-resolver-api-1.9.24.jar (157 kB at 413 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.9.1/asm-9.9.1.jar (126 kB at 381 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/9.9.1/asm-commons-9.9.1.jar (75 kB at 227 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/9.9.1/asm-tree-9.9.1.jar (52 kB at 157 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.6.0/commons-exec-1.6.0.jar (69 kB at 338 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.24/maven-resolver-util-1.9.24.jar (196 kB at 1.7 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.24/maven-resolver-api-1.9.24.jar (157 kB at 413 kB/s)
Servidor gRPC iniciado en puerto 50051
```

Sin cerrar el servidor, se abre una segunda terminal en VS Code y se ejecuta. Después El cliente prueba secuencialmente los tres tipos de llamada:

Llamada Unaria: Envía un mensaje con timestamp de Lamport y recibe un ACK del servidor.

Server Streaming: Solicita 3 noticias y las recibe una a una con un intervalo de 500 ms entre cada una.

Chat Bidireccional: Envía 3 mensajes en stream y recibe una respuesta por cada uno en tiempo real.

como se muestra en la siguiente imagen

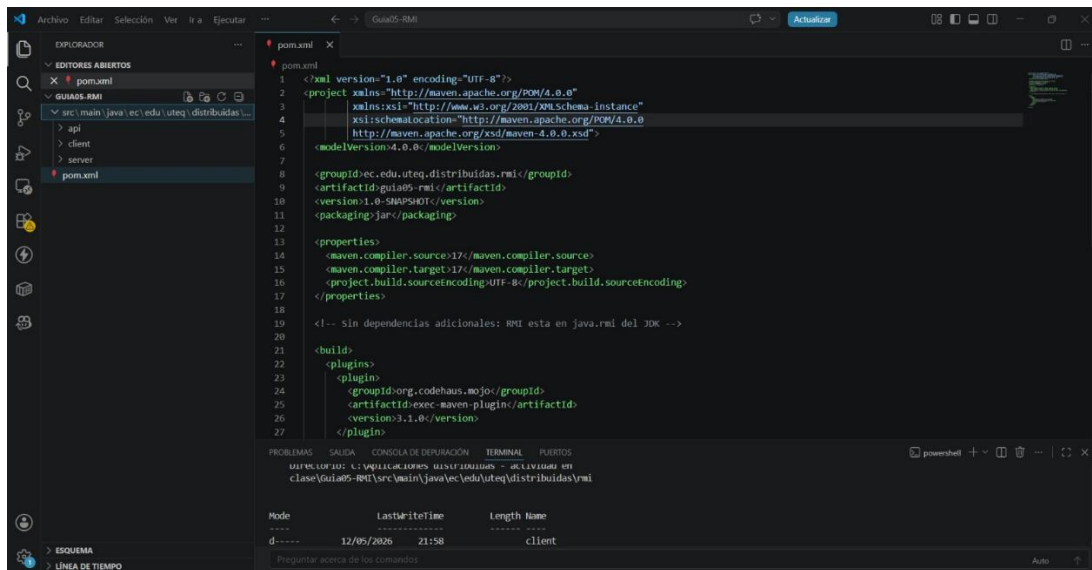
```
Archivo Editar Selección Ver Ir a Ejecutar
pom.xml 1 ClienteGRPC.java X Mensajeria.proto MensajeriaServiceImpl.java 9+ ServidorGRPC.java 1
EXPLORADOR EDITORES ABIERTOS
pom.xml 1
X ClienteGRPC.java 11
Mensajeria.proto 15
MensajeriaS... 9+
ServidorGRPC... 1
GUIA04-GRPC
.github
src\main
java\ec\edu\...
client
ClienteGRPC.java
server
MensajeriaS... 9+
ServidorGRPC... 1
proto
Mensajeria.proto
target
pom.xml 1
ESQUEMA
LÍNEA DE TIEMPO
JAVA PROJECTS
PROBLEMAS 11 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Aplicaciones distribuidas - actividad en clase\Guia04-grpc> mvn exec:java "-Dexec.mainClass=ec.edu.uteq.distribuidas.grpc.client.ClienteGRPC"
[INFO] --- exec:3.6.3:java (default-cli) @ guia04-grpc ---
=== 1. Llamada Unaria ===
Respuesta: ACK: Hola desde gRPC (ts=2)

=== 2. Server Streaming (3 noticias) ===
<< Noticia #1 para clienteA
<< Noticia #2 para clienteA
<< Noticia #3 para clienteA

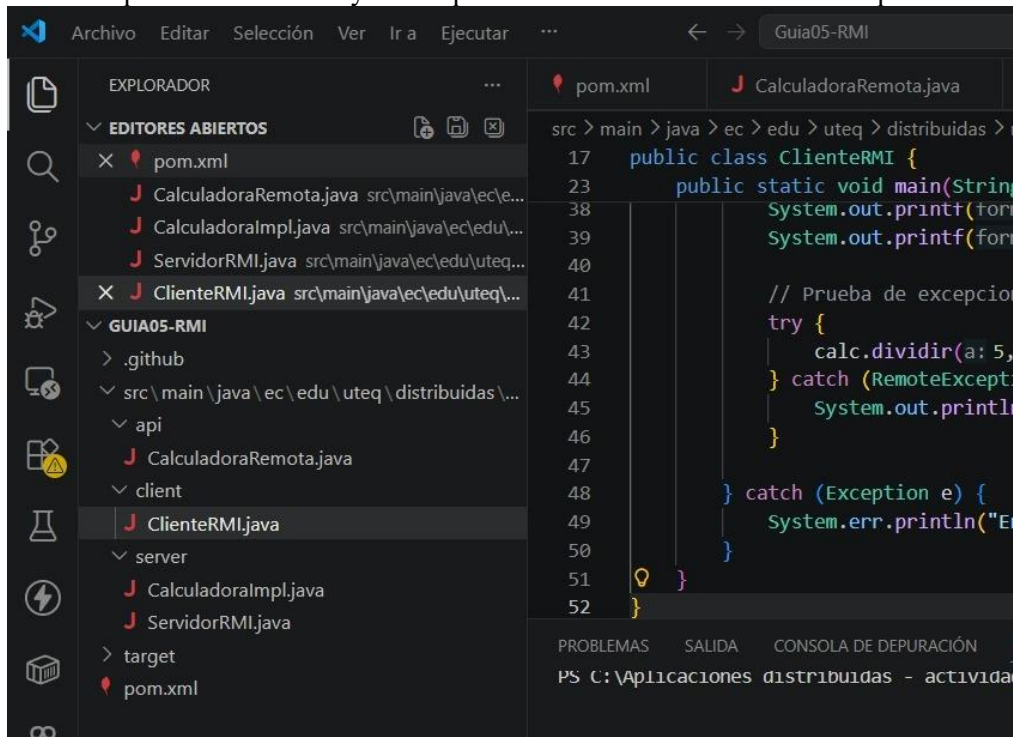
=== 3. Chat Bidireccional ===
<< [clienteA] Mensaje bidireccional #1
<< [clienteA] Mensaje bidireccional #2
<< [clienteA] Mensaje bidireccional #3
Stream completado.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.356 s
[INFO] Finished at: 2026-05-12T21:48:57-05:00
[INFO] -----
PS C:\Aplicaciones distribuidas - actividad en clase\Guia04-grpc>
```

PROYECTO 5

Primero crear el proyecto Maven, despues configurar el pom.xml y crear las carpetas del proyecto.



Para la creación de la estructura de paquetes del proyecto, se organizaron las clases en tres directorios clave que se pueden observar en el explorador que aparece en la imagen. El paquete api contiene la interfaz CalculadoraRemota.java, que actúa como el contrato compartido; el paquete server aloja la implementación lógica en CalculadoraImpl.java y el arranque en ServidorRMI.java; y el paquete client contiene a ClienteRMI.java. Esta separación asegura que el cliente interactúe únicamente con la interfaz, manteniendo total transparencia de acceso y desacoplamiento frente a los detalles de implementación del servidor.



Para la ejecución del proyecto, primero se inicia el servidor, el cual crea el RMI Registry en el puerto 1099, exporta el objeto remoto y permanece a la espera de conexiones entrantes. Posteriormente, desde una segunda terminal se ejecuta el cliente, que emplea Naming.lookup() para obtener el *stub* del objeto remoto y realizar las operaciones de la calculadora de forma transparente a través de la red.

```
[INFO] -----
[INFO] Total time: 1.831 s
[INFO] Finished at: 2026-05-12T22:06:16-05:00
[INFO] -----
PS C:\Aplicaciones distribuidas - actividad en clase\Guia05-RMI> mvn exec:java "-Dexec.mainClass=ec.edu.uteq.distribuidas.rmi.server.ServidorRMI"
[INFO] Scanning for projects...
[INFO] -----< ec.edu.uteq.distribuidas.rmi:guia05-rmi >-----
[INFO] Building guia05-rmi 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- exec:3.1.0:java (default-cli) @ guia05-rmi ---
RMI Registry creado en puerto 1099
Objeto registrado en: rmi://localhost:1099/CalculadoraDistribuida
Servidor RMI listo. Esperando clientes...
Servidor RMI: UTEQ-Servidor-01 | JVM: 17 | Hilos activos: 1
```

Ejecutar el Cliente RMI

El cliente realiza Naming.lookup(url) para obtener un stub (proxy local) del objeto remoto. A partir de ese momento, cada llamada al stub viaja transparentemente por la red como si fuera una llamada local.

Se prueban todas las operaciones de la calculadora como podemos ver en la imagen de ejecución

La última línea demuestra el manejo de excepciones remotas: cuando el servidor lanza una RemoteException por división entre cero, esta viaja serializada por la red y se recibe en el cliente como una excepción normal de Java.

```
[INFO] --- exec:3.1.0:java (default-cli) @ guia05-rmi ---
Conectado a: Servidor RMI: UTEQ-Servidor-01 | JVM: 17 | Hilos activos: 2
-----
10 + 5 = 15,00
10 - 5 = 5,00
10 x 5 = 50,00
10 / 4 = 2,5000
2 ^ 10 = 1024
sqrt(2) = 1,414214
Excepcion remota: RemoteException occurred in server thread; nested exce
ption is:
    java.rmi.RemoteException: Division por cero no permitida
```

En la terminal del servidor se puede observar el registro de cada operación invocada remotamente, para simular dos equipos distintos en red: basta con cambiar la constante HOST en ClienteRMI.java de "localhost" a la IP real del servidor.

```
92 }
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
exec:java "-Dexec.mainClass=ec.edu.uteq.distribuidas.rmi.server.ServidorRMI"
[INFO] Building guia05-rmi 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- exec:3.1.0:java (default-cli) @ guia05-rmi ---
RMI Registry creado en puerto 1099
Objeto registrado en: rmi://localhost:1099/CalculadoraDistribuida
Servidor RMI listo. Esperando clientes...
Servidor RMI: UTEQ-Servidor-01 | JVM: 17 | Hilos activos: 1
[RMI] sumar(10,00, 5,00) invocado
[RMI] restar(10,00, 5,00) invocado
[RMI] multiplicar(10,00, 5,00) invocado
[RMI] dividir(10,00, 4,00) invocado
[RMI] dividir(5,00, 0,00) invocado
[INFO] -----[ jar ]-----
[INFO] --- exec:3.1.0:java (default-cli) @ guia05-rmi ---
Conectado a: Servidor RMI: UTEQ-Servidor-01 | JVM: 17 | Hilos activos: 2
-----
10 + 5 = 15,00
10 - 5 = 5,00
10 x 5 = 50,00
10 / 4 = 2,5000
2 ^ 10 = 1024
sqrt(2) = 1,414214
Excepcion remota: RemoteException occurred in server thread; nested exce
ption is:
    java.rmi.RemoteException: Division por cero no permitida
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

Repositorio de los proyectos:

<https://github.com/BedonViteri/Aplicaciones-Distribuidas.git>



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIA DE LA COMPUTACIÓN Y DISEÑO DIGITAL

CARRERA:
SOFTWARE

DOCENTE:
ING. GUERRERO ULLOA GLEISTON CICERON

ASIGNATURA:
APLICACIONES DISTRIBUIDAS

TEMA:
FORM: FORM-TL-01 — Taller — Modelado de Comunicación
entre Procesos Distribuidos

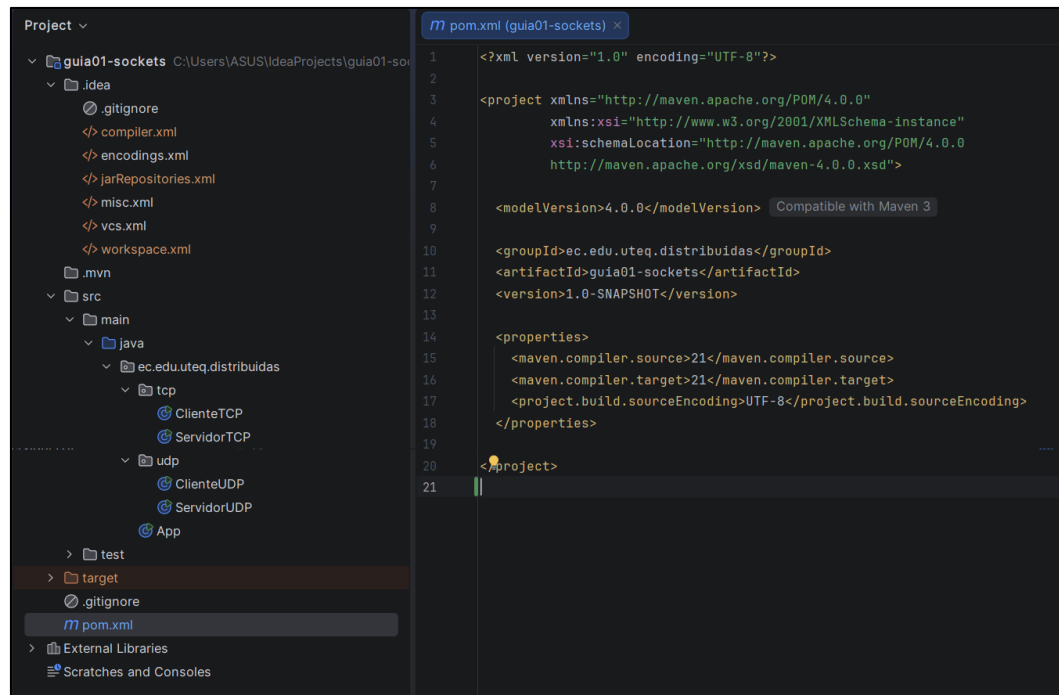
Descripción:
Modalidad: Grupal (equipos de 3-4) | Carácter: Formativa
(preparatoria para PE-U1) Taller práctico en clase que prepara al
equipo para implementar la comunicación entre procesos en la
Práctica Experimental de la Unidad 1.

ESTUDIANTE:
VINUEZA SANCHEZ HAROLD NICOLAS

AÑO LECTIVO
2025 – 2026 PPA

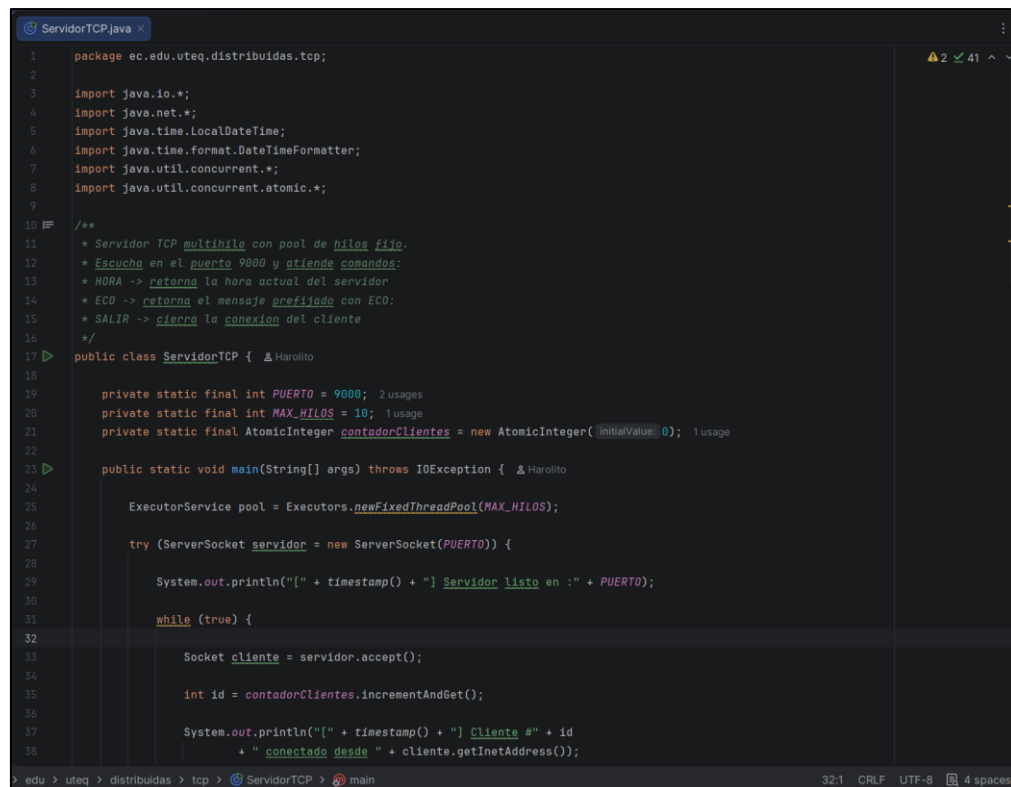
1. Configuración del archivo pom.xml.

En este primer paso se realizó la configuración del archivo pom.xml dentro del proyecto Maven en IntelliJ IDEA. Se establecieron los datos principales del proyecto y la versión de Java 21 para trabajar correctamente durante la práctica.



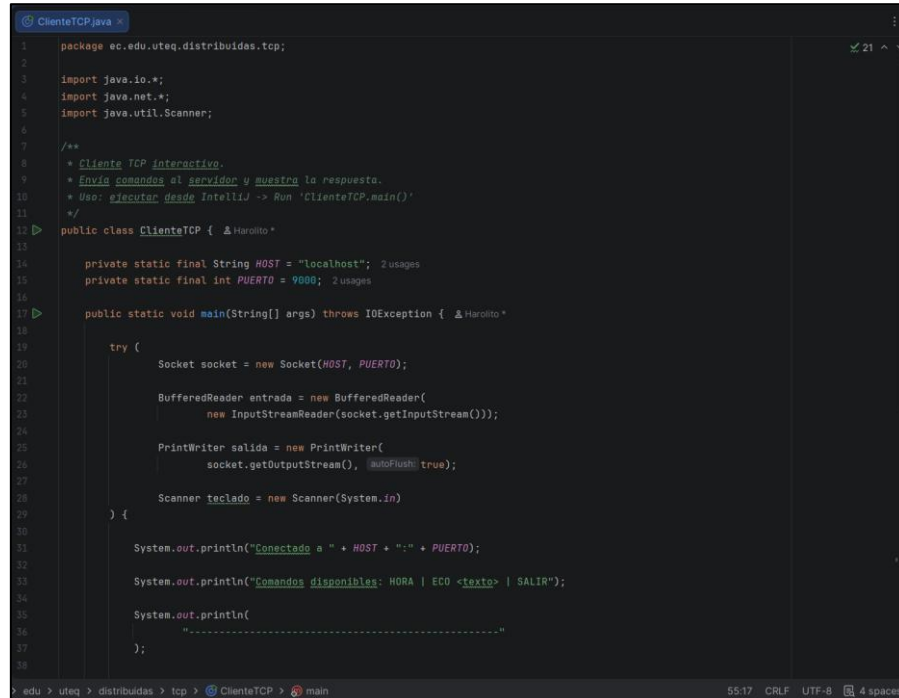
2. Implementación del servidor TCP.

Se desarrolló la clase ServidorTCP, encargada de establecer la comunicación mediante sockets TCP en el puerto 9000. Además, se implementó un sistema multihilo utilizando ExecutorService para permitir la atención simultánea de varios clientes. El servidor también fue configurado para responder comandos como HORA, ECO y SALIR.



3. Implementación del cliente TCP.

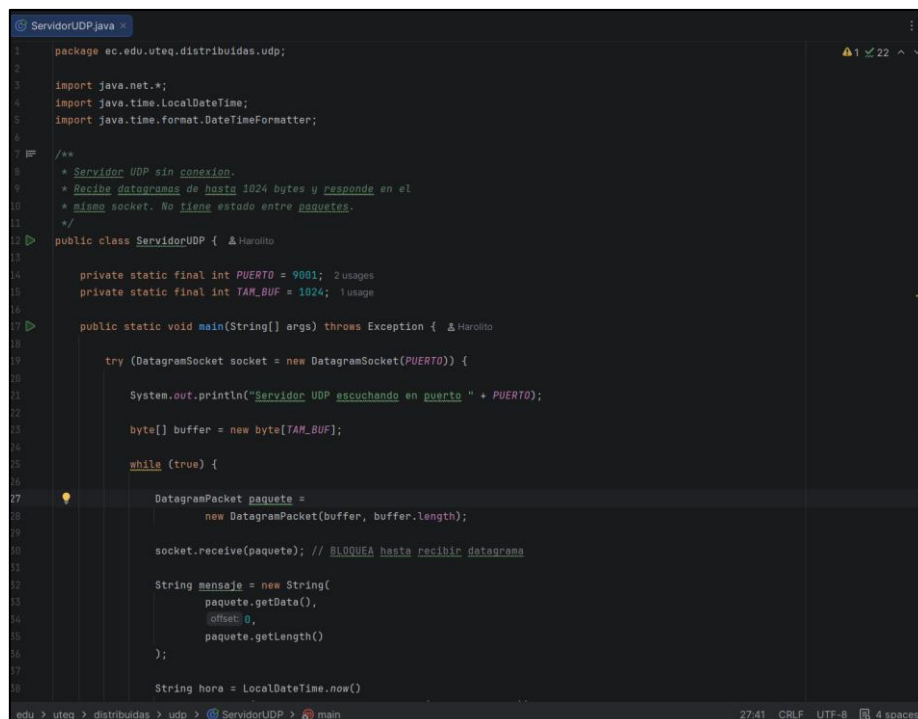
En este paso se desarrolló la clase ClienteTCP, encargada de conectarse al servidor mediante el puerto 9000 utilizando localhost. El cliente fue diseñado para enviar comandos desde consola y mostrar las respuestas recibidas por parte del servidor. Además, se implementó la interacción básica con los comandos HORA, ECO y SALIR.



```
1 package ec.edu.uteq.distribuidas.tcp;
2
3 import java.io.*;
4 import java.net.*;
5 import java.util.Scanner;
6
7 /**
8  * Cliente TCP interactivo.
9  * Envía comandos al servidor y muestra la respuesta.
10  * Uso: ejecutar desde IntelliJ -> Run 'ClienteTCP.main()'
11  */
12 public class ClienteTCP {
13     private static final String HOST = "localhost";
14     private static final int PUERTO = 9000;
15
16     public static void main(String[] args) throws IOException {
17         try {
18             Socket socket = new Socket(HOST, PUERTO);
19             BufferedReader entrada = new BufferedReader(
20                 new InputStreamReader(socket.getInputStream()));
21             PrintWriter salida = new PrintWriter(
22                 socket.getOutputStream(), true);
23             Scanner teclado = new Scanner(System.in);
24
25             System.out.println("Conectado a " + HOST + ":" + PUERTO);
26             System.out.println("Comandos disponibles: HORA | ECO <texto> | SALIR");
27             System.out.println("-----");
28         }
29     }
30 }
```

4. Implementación del servidor UDP.

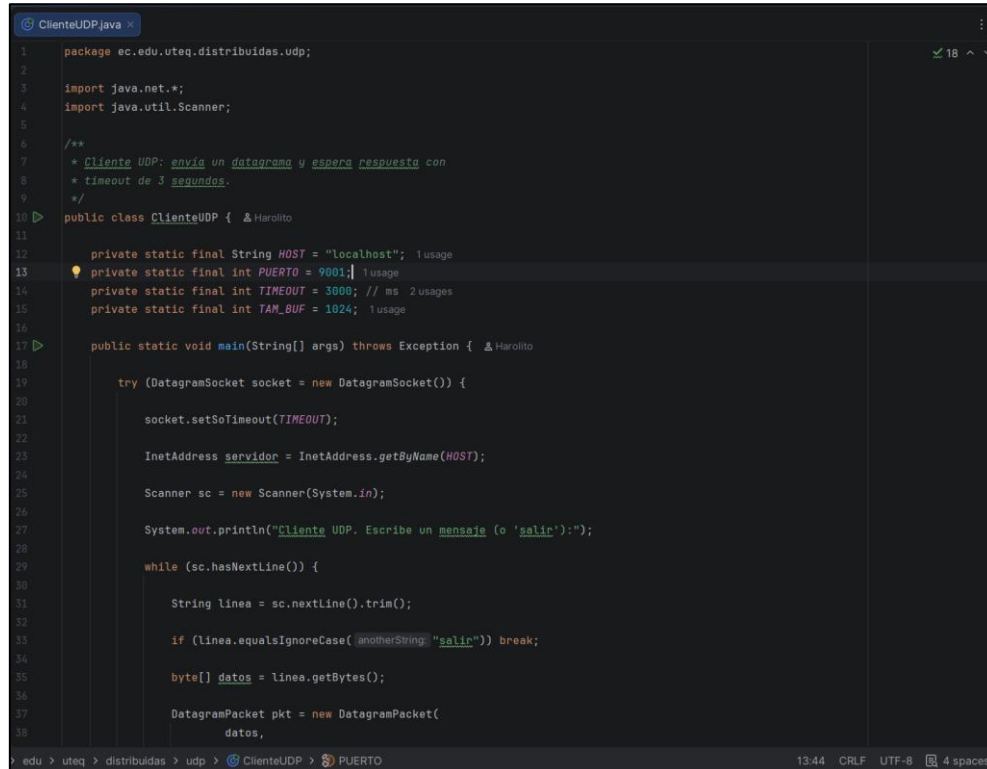
Se desarrolló la clase ServidorUDP, encargada de recibir y responder mensajes mediante datagramas UDP utilizando el puerto 9001. A diferencia de TCP, este protocolo trabaja sin conexión permanente entre cliente y servidor. Además, el servidor fue configurado para recibir mensajes y responder con una confirmación tipo ACK.



```
1 package ec.edu.uteq.distribuidas.udp;
2
3 import java.net.*;
4 import java.time.LocalDateTime;
5 import java.time.format.DateTimeFormatter;
6
7 /**
8  * Servidor UDP sin conexión.
9  * Recibe datagramas de hasta 1024 bytes y responde en el
10  * mismo socket. No tiene estado entre paquetes.
11  */
12 public class ServidorUDP {
13     private static final int PUERTO = 9001;
14     private static final int TAM_BUF = 1024;
15
16     public static void main(String[] args) throws Exception {
17         try (DatagramSocket socket = new DatagramSocket(PUERTO)) {
18             System.out.println("Servidor UDP escuchando en puerto " + PUERTO);
19
20             byte[] buffer = new byte[TAM_BUF];
21
22             while (true) {
23                 DatagramPacket paquete =
24                     new DatagramPacket(buffer, buffer.length);
25
26                 socket.receive(paquete); // BLOQUEA hasta recibir datagrama
27
28                 String mensaje = new String(
29                     paquete.getData(),
30                     0,
31                     paquete.getLength());
32
33                 String hora = LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"));
34             }
35         }
36     }
37 }
```

5. Implementación del cliente UDP.

Se desarrolló la clase ClienteUDP, encargada de enviar mensajes al servidor mediante datagramas UDP utilizando el puerto 9001. Además, se implementó un tiempo de espera de 3 segundos para controlar posibles pérdidas de respuesta durante la comunicación. Finalmente, el cliente permite enviar mensajes desde consola y visualizar la respuesta recibida por parte del servidor.



```
1 package ec.edu.uteq.distribuidas.udp;
2
3 import java.net.*;
4 import java.util.Scanner;
5
6 /**
7  * Cliente UDP: envia un datagrama y espera respuesta con
8  * timeout de 3 segundos.
9  */
10 public class ClienteUDP { &Harolito
11
12     private static final String HOST = "localhost"; 1usage
13     private static final int PUERTO = 9001; 1usage
14     private static final int TIMEOUT = 3000; // ms 2usages
15     private static final int TAM_BUF = 1024; 1usage
16
17     public static void main(String[] args) throws Exception { &Harolito
18
19         try (DatagramSocket socket = new DatagramSocket()) {
20
21             socket.setSoTimeout(TIMEOUT);
22
23             InetAddress servidor = InetAddress.getByName(HOST);
24
25             Scanner sc = new Scanner(System.in);
26
27             System.out.println("Cliente UDP. Escribe un mensaje (o 'salir'):");
28
29             while (sc.hasNextLine()) {
30
31                 String linea = sc.nextLine().trim();
32
33                 if (linea.equalsIgnoreCase("salir")) break;
34
35                 byte[] datos = linea.getBytes();
36
37                 DatagramPacket pkt = new DatagramPacket(
38                     datos,
```

6. Pruebas de funcionamiento del sistema TCP.

Finalmente, se realizaron las pruebas de comunicación entre el cliente y el servidor TCP utilizando comandos desde consola. Durante la ejecución se verificó correctamente el funcionamiento de las instrucciones HORA, ECO y SALIR, permitiendo comprobar el intercambio de mensajes y el cierre adecuado de la conexión entre ambas aplicaciones.



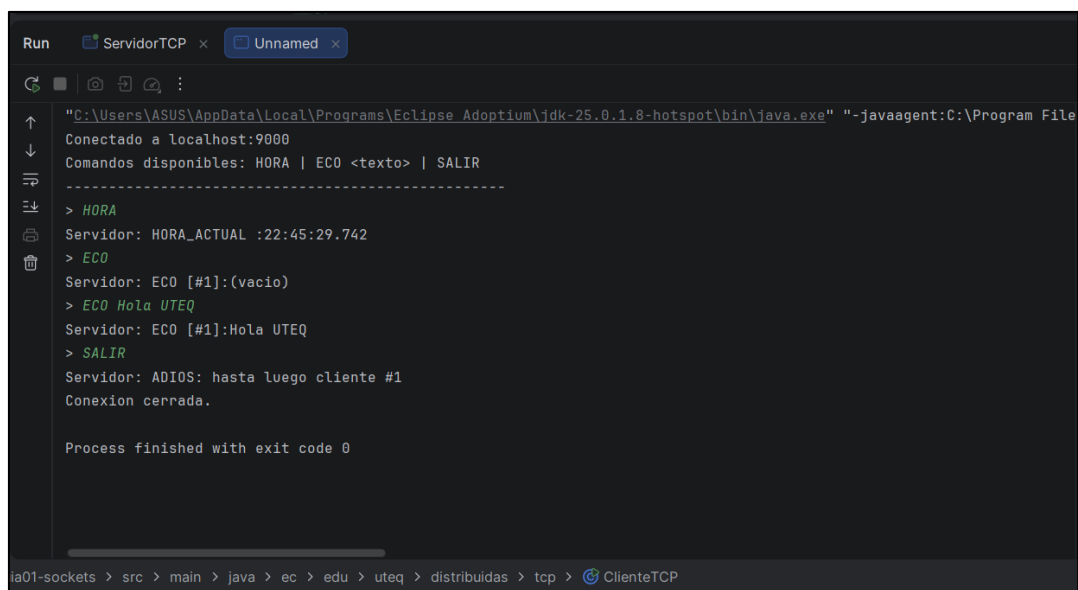
```
Run  ServidorTCP x Unnamed x
C:\Users\ASUS\AppData\Local\Programs\Eclipse Adoptium\jdk-25.0.1-hotspot\bin\java.exe "-javaagent:C:\Program File
Conectado a localhost:9000
Comandos disponibles: HORA | ECO <texto> | SALIR
-----
> HORA
Servidor: HORA_ACTUAL :22:35:49.843
>
```



```
Run  ServidorTCP x Unnamed x
C:\Users\ASUS\AppData\Local\Programs\Eclipse Adoptium\jdk-25.0.1.8-hotspot\bin\java.exe "-javaagent:C:\Program File
Conectado a localhost:9000
Comandos disponibles: HORA | ECO <texto> | SALIR
-----
> HORA
Servidor: HORA_ACTUAL :22:35:49.843
> ECO
Servidor: ECO [#1]:(vacio)
> SALIR
Servidor: ADIOS: hasta luego cliente #1
Conexion cerrada.

Process finished with exit code 0

la01-sockets > src > main > java > ec > edu > uteq > distribuidas > tcp > ClienteTCP
```



```
Run  ServidorTCP x Unnamed x
C:\Users\ASUS\AppData\Local\Programs\Eclipse Adoptium\jdk-25.0.1.8-hotspot\bin\java.exe "-javaagent:C:\Program File
Conectado a localhost:9000
Comandos disponibles: HORA | ECO <texto> | SALIR
-----
> HORA
Servidor: HORA_ACTUAL :22:45:29.742
> ECO
Servidor: ECO [#1]:(vacio)
> ECO Hola UTEQ
Servidor: ECO [#1]:Hola UTEQ
> SALIR
Servidor: ADIOS: hasta luego cliente #1
Conexion cerrada.

Process finished with exit code 0

la01-sockets > src > main > java > ec > edu > uteq > distribuidas > tcp > ClienteTCP
```

7. Link del repositorio de GitHub.

<https://github.com/Harold-Vinueza/Gu-a-1---APLICACIONES-DISTRIBUIDAS>

8. Conclusión.

Durante el desarrollo de la práctica se logró comprender el funcionamiento de la comunicación cliente-servidor mediante sockets TCP y UDP utilizando Java. Además, se fortalecieron conocimientos relacionados con el uso de puertos, envío de mensajes y manejo de conexiones dentro de aplicaciones distribuidas. Finalmente, se identificaron las diferencias principales entre el protocolo TCP, orientado a conexión, y UDP, basado en comunicación sin conexión, permitiendo obtener una visión más clara sobre el comportamiento de ambos modelos de comunicación.